Updatable Block-Level Message-Locked Encryption

Yongjun Zhao, *Member, IEEE* and Sherman S. M. Chow, *Member, IEEE*

Abstract—Deduplication is widely used for reducing the storage requirement for storage service providers. Nevertheless, it is unclear how to support deduplication of encrypted data securely until the study of Bellare et al. on message-locked encryption (MLE, Eurocrypt 2013). While updating (shared) files is natural, existing MLE solutions do not allow efficient update of encrypted files stored remotely. Even modifying a single bit requires the expensive way of downloading and decrypting a large ciphertext (then re-uploading). This paper initiates the study of updatable block-level MLE, a new primitive in incremental cryptography and cloud cryptography. Our proposed provably-secure construction is updatable with computation cost logarithmic in the file size. It naturally supports block-level deduplication. It also supports proof-of-ownership which protects storage providers from being abused as a free content distribution network. Our experiments show its practical performance relative to the original MLE and existing non-updatable block-level MLE.

Index Terms—Message-locked encryption, proofs-of-ownership, deduplication, hash function, incremental cloud cryptography

1 Introduction

Deduplication, which eliminates redundant copies in user-provided data, has been widely used to improve storage utilization and reduce communication cost. The saving is significant for (cloud) storage provider which stores data from many clients. Typically, storage systems require seeing the client data in plaintext to perform deduplication, which incurs an obvious privacy threat. On the other hand, using a user-specific key to encrypt the data before upload drastically reduces the effectiveness of deduplication since the probabilistic nature of secure encryption intuitively produces random-looking ciphertexts. As far as we know, the first attempt to resolve the seemingly contradicting requirements (of achieving file confidentiality and file deduplication simultaneously) was *convergent encryption* [2]. It works by deterministically deriving a key for each file by hashing the file and then encrypting each using such a file-derived key. This procedure guarantees that identical files are encrypted identically, enabling efficient deduplication.

Convergent encryption (and its variants) has been used in numerous applications [3], [4], [5]. However, the precise security guarantee provided by convergent encryption is unknown until the formulation of message-locked encryption (MLE) by Bellare et al. [6]. MLE is a symmetric encryption scheme in which the message is the lock, i.e., it uses the message to derive the key for encryption and decryption. Their study provided several security definitions and analyzed some prior schemes under their framework.

While earlier MLE [6], [7], [8] focus on *file-level deduplication*, a study [9] has shown that *block-level deduplication* can be more space-efficient. Naturally, recent research extended file-level deduplication on encrypted data to the block-level setting [10], [11]. The most straightforward solution is to apply file-level MLE on each block independently. Unfortunately, this approach increases the number of keys for file F from $O(1)$ to $O(|F|/B)$ for block size B, which is unsatisfactory. In general, we want to minimize the complexity of key management and avoid using any additional key management server. Jiang et al.'s deduplication protocol [11] only supports intra-user block-level deduplication. Chen et al. [10] formalized *block-level message-locked encryption* (BL-MLE) and designed the first solution that is space-efficient, supporting both file-level and block-level (cross-user) deduplication simultaneously (called dual-level deduplication). However, it is pairing-based and is relatively slow.

Unfortunately, all existing works focus on *static files*, with no support of efficient file update (even with the help of a key-management server). To modify a single bit, the file owner has to download the whole encrypted file, decrypt, update, re-encrypt, and then upload the new ciphertext to the cloud. The computation and communication costs of all these operations are linear in the file size, i.e., $O(|F|)$, which are too expensive for large files. In many applications, we need to incrementally update a large file that can be deduplicated across different users from time to time. For instance, astronomers update high-resolution images of different parts of the universe periodically [12]; administrators or users may back up virtual machine images [13] regularly.

We thus ask ourselves a practically motivated question:

"Can the update cost of MLE be sublinear?"

Update becomes nontrivial in the MLE/BL-MLE context. The difficulty arises because a typical MLE/BL-MLE scheme derives the encryption key from the file deterministically. A single bit flip will lead to a totally different encryption key, thus the whole file needs to be re-encrypted. One

• *The authors are with the Department of Information Engineering, The Chinese University of Hong Kong, Hong Kong. E-mail: {yjzhao, sherman}@ie.cuhk.edu.hk.*

Manuscript received 7 Aug. 2018; revised 29 May 2019; accepted 4 June 2019. Date of publication 12 June 2019; date of current version 9 July 2021. (Corresponding author: Sherman S. M. Chow.) Digital Object Identifier no. 10.1109/TDSC.2019.2922403

1545-5971 © 2019 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

may try to remedy such inefficiency by independently applying (file-level) MLE on each block. But as mentioned above, this approach introduces a large number of block keys. Bellare and Keelveedhi proposed *interactive MLE* (iMLE) [8], while a standard MLE does not require interaction. Their solution is the first MLE supporting incremental updates. However, their scheme only deduplicates at the file level. In a nutshell, updatable block-level MLE remains open.

1.1 Incremental Cloud Cryptography

Bellare et al. initiated the research of incremental cryptography, which aims to update the result of a cryptographically processed object faster than recomputing from scratch. Early works focus on signing and hashing [14], [15], while the follow-ups studied encryption [16], [17]. Recently, we witnessed a growing interest in cloud cryptography, e.g., searchable encryption, message-locked encryption, and proof of retrievability. There are also studies on dynamic searchable encryption [18], [19], [20] where searchable ciphertext can be updated without rebuilding from scratch.

We further this research line for cloud cryptography. Our paper initiates the study of incremental cryptography for secure (block-level) deduplication, which is of practical necessity due to the wide application of cloud services.

1.2 Our Contribution

This paper makes the following five contributions.

1) We initiate the study of efficiently updatable[1] block-level message-locked encryption (UMLE) which naturally supports block-level deduplication. Compared with our conference version [1], we revise the security notion of UMLE/BL-MLE (in Section 3.4.1), in particular, tag consistency for both file tag and block tags. Only file-tag consistency is defined before [1]. The tag consistency notion of Chen et al. [10] is only for block tags. Instead of re-using theirs, we define a new version of block-tag consistency.

2) Our theoretical contribution leads to practical consequences. We revisit Chen et al.'s (pairing-based) BL-MLE construction [10], and show that it does not satisfy their tag consistency notion as claimed (and hence it is vulnerable to duplication-faking attack, also see Section 3.4.1).

3) We propose an efficient (pairing-free) UMLE construction in the random oracle model [1], with revised security analysis showing that it satisfies our revised definition. Our construction is thus the only UMLE/BL-MLE scheme that satisfies tag consistency. For a file of size $|F|$, the update cost of our UMLE construction is only $O(\log|F|)$ where the base of the logarithm is B/λ (typically a large constant), B is the block size, and λ is the key size.

4) Borrowing ideas from the database community, we propose an extension that also supports block insertion and deletion, with an amortized complexity of $O(\log|F|)$.

5) We further implement our UMLE construction and BL-MLE [10]. We provide a comprehensive performance evaluation for both and a comparison among them. Chen et al. [10] only reported the performance of two algorithms in BL-MLE. Our evaluation not only supplements their report, but also shows that our construction is computationally more efficient than theirs. This holds even without considering the new update function and the corresponding efficiency gain over the naïve approach of updating the whole file. We also compare with convergent encryption version of file-level MLE, which shows that UMLE has comparable performance.

Simply put, the efficiency gain of our block-level update protocol does not heavily penalize the performance of "traditional" (block/file-level) MLE operations.

As a side-product, we implement incremental hashing by Bellare and Micciancio [15], a building block of UMLE. We evaluate its computational performance under different parameters. To our knowledge, such evaluation is the first in the literature and will be useful for other applications.

1.3 Paper Organization

Section 2 introduces our notions and technical preliminaries. Section 3 defines the interfaces of UMLE and its security definitions. Our proposed construction is described in Section 4. Section 5 provides a comprehensive performance evaluation. We survey related works in Section 6, especially those subsequent to our conference version.

Supplementary material,[2] provides security proofs of our construction, followed by a discussion on how to extend our construction to support insertion and deletion. We also present a performance evaluation of the multiplicative incremental hash by Bellare and Micciancio [15] and an attack against the scheme of Chen et al. [10].

2 Preliminary

2.1 Notations

For $i \in \mathbb{N}$, we let $[i] = \{1, \ldots, i\}$. For vector $\mathbf{x}$, $|\mathbf{x}|$ denotes its length, $\mathbf{x}[i]$ denotes the i-th component, and for $1 \le i \le j \le |\mathbf{x}|$, $\mathbf{x}[i,j] = \mathbf{x}[i] \cdots \mathbf{x}[j]$. If S is a finite set, $|S|$ denotes its size and $s \xleftarrow{\$} S$ means uniformly picking an element in S and assigning it to s. By $y \xleftarrow{\$} A(x_1, \ldots)$, we mean running algorithm A on inputs $x_1, \ldots$ with random coins.

The guessing probability $\mathbf{GP}(X)$ and min-entropy $\mathbf{H}_\infty(X)$ of a random variable X are defined by $\mathbf{GP}(X) = \max_x \Pr[X = x] = 2^{-\mathbf{H}_\infty(X)}$. The conditional guessing probability $\mathbf{GP}(X|Y)$ and conditional min-entropy $\mathbf{H}_\infty(X|Y)$ of X given a random variable Y are defined by $\mathbf{GP}(X|Y) = \sum_y \Pr[Y = y] \cdot \max_x \Pr[X = x | Y = y] = 2^{-\mathbf{H}_\infty(X|Y)}$.

2.2 Unpredictable Sources

A source is a polynomial-time algorithm $\mathcal{M}$ that on input 1^λ returns $(\mathbf{M}, Z)$ where $\mathbf{M}$ is a message vector in $\{0,1\}^*$ and $Z \in \{0,1\}^*$ denotes some auxiliary information. Let $n(\lambda)$ be the vector length. In our context, it is the number of blocks. For all $i \in [1, n(\lambda)]$, $\mathbf{M}[i]$ represents the i-th block of the message $\mathbf{M}$. We restrict our attention to sources that output message vector over $\{0,1\}^B$, where B is the block size. So $\mathbf{M}[i] \in \{0,1\}^B, \forall i$. Similar to previous works [6], [10] we require that $\mathbf{M}[i_1] \neq \mathbf{M}[i_2]$ for all distinct $i_1, i_2 \in [m(\lambda)]$ to bar against a trivial adversary. We say a source $\mathcal{M}$ is unpredictable if $\mathbf{GP}_{\mathcal{M}} = \max_i\{\mathbf{GP}(\mathbf{M}[i]|Z)\}$ is negligible.

1. We did not name it incremental MLE following incremental hash of Bellare et al. [14] to avoid clashing with the acronym iMLE used by Bellare and Keelveedhi [8] for interactive MLE.

2. which can be found on the Computer Society Digital Library at http://doi.ieeecomputersociety.org/10.1109/TDSC.2019.2922403

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

2.3 Message-Locked Encryption

MLE consists of five algorithms [6]. Given a security parameter 1^λ, the parameter generation algorithm Setup returns a public parameter P. On input[3] a message $\mathbf{M}$, the key generation algorithm KeyGen returns a message-derived key $K \xleftarrow{\$} \mathsf{KeyGen}(\mathbf{M})$. On inputs $K, \mathbf{M}$, the encryption algorithm Enc returns a ciphertext $\mathbf{C} \xleftarrow{\$} \mathsf{Enc}(K, \mathbf{M})$. With $\mathbf{C}$, a tag $\mathbf{T} \xleftarrow{\$} \mathsf{TagGen}(\mathbf{C})$ can be generated by the tag generation algorithm. On inputs $K, \mathbf{C}$, the decryption algorithm returns $\mathsf{Dec}(K, \mathbf{C}) \in \{0,1\}^* \cup \{\perp\}$, where $\perp$ denotes invalid. All algorithms except Dec can be probabilistic. MLE is deterministic if both KeyGen and Enc are deterministic.

2.4 Proof of Ownership

Deduplication can take place on the server side or the client side. Consider a simple client-side deduplication mechanism which uses the hash of a file for "deduplication" and "ownership-proof." It reduces the communication cost but also opens up opportunities for malicious activities. A malicious user can "claim ownership" of a confidential file uploaded by another user even if only a short hash value is somehow leaked to her. Another attack is to publicize a short hash value of a huge file (say a movie) so that everyone who just downloaded the hash can easily claim the ownership of and hence download the file, effectively making the cloud as a cheap content distribution center.

Proof-of-ownership (PoW) protects both storage servers and cloud users from these attacks [21]. It allows a prover (file owner) to convince the verifier (storage server) that she indeed owns a file in its entirety, but not just some "leakages." For efficiency, verification does not require accessing the file which is stored in a relatively slow storage medium.

We briefly describe the idea behind three existing PoW protocols [21] with different trade-offs. They require both the user and the server to build a Merkle-tree on a buffer derived from a pre-processed file. The server only keeps the root and challenges the client to present valid sibling paths for a subset of leaves of the tree. Therefore, the bandwidth cost would be a super-logarithmic number of sibling paths of the Merkle-tree. Also, verification does not require the file at all but just a hash value. There are other more involved techniques to build PoW which tolerates unbounded leakage [22] as long as the original file cannot be efficiently computable from the leakage. For this strong security level, verification requires the original file.

2.5 Deterministic Symmetric-Key Encryption

A deterministic symmetric-key encryption scheme consists of the following algorithms (SKE.KeyGen, SKE.Enc, SKE.Dec), and only SKE.KeyGen is randomized. We require it to provide both key recovery (KR) security and one-time real-or-random (ROR) security [6]. A blockcipher in counter mode with a fixed IV satisfies these requirements.

The key recovery game is defined as follows. On input 1^λ, the challenger chooses a randomly chosen key K. The adversary can query at most once to an encryption oracle a message M. The challenger replies with $c = \mathsf{SKE.Enc}(K, M)$. The adversary wins the game if it can output $K' = K$. We say that a symmetric-key encryption scheme is KR-secure if the advantage $\mathbf{Adv}^{\mathcal{A}}_{\mathsf{KR}}(\lambda)$ of any adversary in winning the above game is negligible.

The real-or-random game is defined as follows. On input 1^λ, the challenger chooses a random bit b. The adversary can make multiple queries to an encryption oracle, each time for a different message $M \in \{0,1\}^*$. For each query, a fresh random key K is chosen. The challenger returns $c = \mathsf{SKE.Enc}(K, M)$ if $b = 1$; otherwise, it chooses a random message $R \in \{0,1\}^{|M|}$ of the same length as M and returns $c = \mathsf{SKE.Enc}(K, R)$. The adversary wins the game if it can output $b' = b$. We say that a symmetric-key encryption scheme is ROR-secure if the advantage $\mathbf{Adv}^{\mathcal{A}}_{\mathsf{ROR}}(\lambda)$ of any adversary in winning the above game is negligible.

2.6 (Incremental) Hash Functions

Cryptographic hash functions take arbitrary-length strings and map them into fixed-length strings. A family of hash functions is a pair of polynomial-time algorithms: key generation algorithm HashGen and hashing algorithm H. The key generation algorithm takes a security parameter 1^λ as input and outputs a key s. The hashing algorithm returns a string $H_s(m)$ by taking a key s and a message m as inputs.

We often require the hash functions to have *collision resistance*. Informally, it means it is infeasible for any computationally bounded adversary to find $m \neq m'$ such that $H_s(m) = H_s(m')$. For the rest of the paper, we assume that s is chosen by some trusted setup and we omit it for brevity.

Incremental hashing is proposed [14] to allow efficient updates of a hash value. Notably, Bellare and Micciancio [15] constructed a collision-resistant incremental hash function in the random oracle model. Looking ahead, we will use their incremental hash in the file tag generation algorithm in our basic construction. Even with that, we still need a specific way to build the ciphertext and the file tag for efficient insertion and deletion. For completeness, we present their "multiplicative" construction (IH.Hash, IH.Update) below.

Let $\mathbb{G}$ be a multiplicative group such that discrete logarithm problem is hard, and let $\mathsf{H} : \{0,1\}^* \to \mathbb{G}$ be a hash function mapping an arbitrary bit string to a point on the elliptic curve (as a multiplicative group element). For an n-block input $\mathbf{M} = \mathbf{M}[1]||\mathbf{M}[2]||\cdots||\mathbf{M}[n]$, the multiplicative incremental hash function [15] which is provably collision-resistant in the random oracle model is defined by

$$\mathsf{IH.Hash}(\mathbf{M}[1]||\cdots||\mathbf{M}[n]) = \textstyle\prod_{i=1}^{n} \mathsf{H}(\langle i \rangle||\mathbf{M}[i]),$$

where $\langle i \rangle$ is the binary representation of integer index i. The product is taken in $\mathbb{G}$ over which we are working in.

The update algorithm takes as input a block index i, the original block $\mathbf{M}[i]$, the updated version $\mathbf{M}'[i]$, and the original hash value y for the whole message $\mathbf{M}$.

$$\mathsf{IH.Update}(i, \mathbf{M}[i], \mathbf{M}'[i], y) = y \cdot \mathsf{H}(\langle i \rangle||\mathbf{M}[i])^{-1} \cdot \mathsf{H}(\langle i \rangle||\mathbf{M}'[i]),$$

where $(\cdot)^{-1}$ denote the inverse operation of the group $\mathbb{G}$.

3 Updatable Block-Level MLE

We define UMLE, for "(Efficiently) *Updatable* (Block-Level) *Message-Locked Encryption*." Since block-level MLE exists [10] and our definitions also extend those for file-level MLE [6], [7], some key differences will be explained shortly.

3. We assume that P (implicitly includes 1^λ) is the default input for the rest of the algorithms, and thus will be omitted for simplicity.

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

3.1 Syntax

A UMLE scheme is specified by the following algorithms.

Setup: takes a security parameter 1^λ and returns the system parameter P. We assume that P implicitly contains 1^λ, and it will be the default input for the rest of the algorithms.

KeyGen: takes a file message $\mathbf{M} = \mathbf{M}[1]||\cdots||\mathbf{M}[n]$ as input, returns block keys $k_1, \ldots, k_n$ and a master key k_{mas} respectively via the following two sub-algorithms:

- B-KeyGen: takes input $\mathbf{M}[i]$, returns the block key k_i;
- M-KeyGen: inputs $\mathbf{M}$, returns the master key k_{mas}.

Enc: takes a file $\mathbf{M}$ (and block keys $k_1, \ldots, k_n$ which can be generated from KeyGen($\mathbf{M}$)) as input, returns the ciphertext $\mathbf{C}$. It consists of the following two sub-algorithms:

- B-Enc: takes a block message $\mathbf{M}[i]$ and the corresponding block key k_i as input ($1 \le i \le n$), returns the block ciphertext $\mathbf{C}[i]$;
- BK-Enc: takes $k_1, \ldots k_n$ as input only, outputs the encrypted block keys as $\mathbf{C}[n+1]||\cdots||\mathbf{C}[n']$.

After completion of the two sub-algorithms B-Enc and BK-Enc, Enc returns encrypted file as $\mathbf{C}$ where $\mathbf{C} = \mathbf{C}[1]||\cdots||\mathbf{C}[n]||\mathbf{C}[n+1]||\cdots||\mathbf{C}[n']$.

Dec: takes the encrypted file $\mathbf{C} = \mathbf{C}[1]||\cdots||\mathbf{C}[n]||\mathbf{C}[n+1]||\cdots||\mathbf{C}[n']$ and k_{mas} as input, returns $\mathbf{M} = \mathbf{M}[1]||\cdots||\mathbf{M}[n]$. Again it consists of two sub-algorithms:

- BK-Dec: takes the master key k_{mas} and the encrypted block keys $\mathbf{C}[n+1]||\cdots||\mathbf{C}[n']$ as input, outputs the set of block keys $k_1, \ldots, k_n$;
- B-Dec: takes the block key k_i and the corresponding block ciphertext $\mathbf{C}[i]$ as input ($1 \le i \le n$), outputs the file block $\mathbf{M}[i]$.

TagGen: takes an encrypted file $\mathbf{C}$ as input, returns block tags $T_1, \ldots, T_{n'}$ and the file tag T_0 generated using the following two sub-algorithms respectively:

- B-TagGen: takes $\mathbf{C}[i]$ as input, returns the block tag T_i ($1 \le i \le n'$);
- M-TagGen: takes $\mathbf{C}$ as input, returns the file tag T_0.

Update: The user (file owner) takes as inputs k_{mas}, the to-be-updated block index i, and a plaintext block $\mathbf{M}[i]^*$; while the server takes $\mathbf{C}$ as input. After their interaction, the user outputs an updated master key k^*_{mas}; the server outputs the updated ciphertext $\mathbf{C}^*$.

With the notation of two-party computation, the syntax of Update is $(k^*_{\mathsf{mas}}, \mathbf{C}^*) \leftarrow \mathsf{Update}((k_{\mathsf{mas}}, i, \mathbf{M}[i]^*), \mathbf{C})$.

UpdateTag: takes the file tag T_0, block tags $T_1, \ldots, T_{n'}$, the ciphertext $\mathbf{C}$, and the updated version $\mathbf{C}'$ as inputs, returns updated file tag T'_0 and block tags $T'_1, \ldots, T'_{n'}$.

The above algorithms are compulsory for UMLE. The following PoW protocol is optional. (See *Alternative formulations* below for details.)

PoWPrf: takes a challenge $Q \subsetneq [n']$ and a file $\mathbf{M}$ as inputs, the prover returns a response $\mathcal{P}$.

PoWVer: takes $\mathcal{P}$ from the prover, a challenge Q, (a subset of) the block tags $\{T_i\}_{i \in Q}$, returns 1 or 0.

3.2 Discussions of our Formulation

Updating the ciphertext and the tag. We add a new interactive protocol Update between a user and the server that updates the ciphertext, and an UpdateTag algorithm that updates the affected file tag and block tag(s).

We define UpdateTag in a general way which takes all possible inputs. One can instantiate UpdateTag which only touches the tags corresponding to the modified ciphertext, and the computational complexity can be only linear in the difference between $\mathbf{C}$ and $\mathbf{C}^*$, i.e., sublinear in $|\mathbf{C}|$.

Encryption and decryption. In all existing MLE/BL-MLE schemes that we are aware of, k_{mas} is derived from M. In our solution, the Enc algorithm takes M as input only without k_{mas}. The M-KeyGen algorithm (which generates k_{mas}) and the B-Enc algorithms (which is a part of the Enc algorithm) are basically the same algorithm with different outputs. (See Algorithm 1 for details.) Dropping k_{mas} does not affect correctness, nor does it conflict with existing notions. k_{mas} is only generated at the end of B-Enc algorithm (thus is not available at the time when Enc (or more precisely B-Enc) is just executed. Hence, it is necessary to drop k_{mas}.

We split the Enc and Dec algorithms into two phases (in addition to KeyGen and TagGen [10]) for blocks and block keys respectively. The B-Enc/B-Dec algorithm encrypts/decrypts each block $\mathbf{M}[i]/\mathbf{C}[i]$ using block keys k_i, while the BK-Enc/BK-Dec algorithm generates/decrypts encrypted block keys $\mathbf{C}[n+1]||\cdots||\mathbf{C}[n']$.

Alternative formulations. In some literature [10], it is the user who executes the TagGen algorithm, which takes *plaintext* as input. To detect maliciously generated tags by users, Chen et al. [10] introduce a **ConTest**[4] algorithm. In contrast, we assume that the tags (both file tags and block tags) can be computed from *ciphertexts* by the honest server. So we can remove **ConTest** for simplicity.[5] Another advantage of this syntax is that the client can be oblivious in the tag generation and the server-side deduplication process.

While we consider server-side deduplication, we follow Chen et al. [10] to include PoW algorithms for flexibility. A server which decides to use client-side deduplication (say, for reducing communication) can first ask the client to prove the file ownership for better security (see Section 2.4).

Randomized tags are considered in some literature [7], [10], which mandate the need for a specialized **EqTest** algorithm for equality test. For deterministic tags generation as in our definition, **EqTest** is just the normal "=" operation. A more general definition which takes care of randomized tags generation can be obtained by adding back **EqTest** [10].

3.3 Correctness

Every UMLE scheme is associated with a file space $\mathrm{MsgSp}(\lambda)$ and a block space $\mathrm{BlSp}(\lambda)$, both are parameterized by the security parameter $\lambda \in \mathbb{N}$. For all $\lambda \in \mathbb{N}, P \leftarrow \mathsf{Setup}(1^\lambda)$, and all $\mathbf{M} \in \mathrm{MsgSp}(\lambda)$, we require:

- *Decryption Correctness.* For any block message $\mathbf{M}[i] \in \mathrm{BlSp}(\lambda)$, block key $k_i \leftarrow \mathsf{B\text{-}KeyGen}(\mathbf{M}[i])$, and block ciphertext $\mathbf{C}[i] \leftarrow \mathsf{B\text{-}Enc}(k_i, \mathbf{M}[i])$, decryption correctness requires $\mathsf{B\text{-}Dec}(k_i, \mathbf{C}[i]) = \mathbf{M}[i]$.

4. To differentiate, we use boldface when referring algorithms in BL-MLE framework [10] (such as **ConTest** and **EqTest**).

5. If TagGen takes plaintext as input, **ConTest** seems necessary for (file/block)-tag consistency (Section 3.4.1) as in some formulation [10].

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

- *Block-Key Retrieval Correctness*. For all messages $\mathbf{M} \in \mathrm{MsgSp}(\lambda)$, master key $k_{\mathsf{mas}} \leftarrow \mathsf{M\text{-}KeyGen}(\mathbf{M})$, and encrypted block keys $\mathbf{C}[n+1]||\cdots||\mathbf{C}[n']$, we have $\mathsf{BK\text{-}Dec}(k_{\mathsf{mas}}, \mathbf{C}[n+1]||\cdots||\mathbf{C}[n']) = (\mathsf{B\text{-}KeyGen}(\mathbf{M}[1]), \ldots, \mathsf{B\text{-}KeyGen}(\mathbf{M}[n]))$.
- *Tag Correctness*. For any $\mathbf{M}[i]$, $\mathbf{M}'[j] \in \mathrm{MsgSp}(\lambda)$ s.t. $\mathbf{M}[i] = \mathbf{M}'[j]$, then for block tags $T_i \leftarrow \mathsf{B\text{-}TagGen}(\mathsf{B\text{-}Enc}(\mathsf{B\text{-}KeyGen}(\mathbf{M}[i]), \mathbf{M}[i]))$ and $T'_j \leftarrow \mathsf{B\text{-}TagGen}(\mathsf{B\text{-}Enc}(\mathsf{B\text{-}KeyGen}(\mathbf{M}'[j]), \mathbf{M}'[j]))$, $T_i = T'_j$.

  Furthermore, if it holds that $\mathbf{M} = \mathbf{M}'$, we require that all the encrypted block keys (and thus their tags) and the file tags are equal. Namely, $\mathsf{Enc}(\mathbf{M}) = \mathsf{Enc}(\mathbf{M}')$ and $T_i = T'_i$ holds $\forall i : 0 \le i \le n'$.
- *Update Correctness*. For any $\mathbf{M} = \mathbf{M}[1]||\cdots||\mathbf{M}[n] \in \mathrm{MsgSp}(\lambda)$, and any file block $\mathbf{M}[i]^* \in \mathrm{BlSp}(\lambda)$, let $(k^*_{\mathsf{mas}}, \mathbf{C}^*) \leftarrow \mathsf{Update}((k_{\mathsf{mas}}, i, \mathbf{M}[i]^*), \mathbf{C})$ where $k_{\mathsf{mas}} = \mathsf{M\text{-}KeyGen}(\mathbf{M})$ and $\mathbf{C} = \mathsf{Enc}(\mathbf{M})$, we have $\mathsf{Dec}(\mathbf{C}^*) = \mathbf{M}[1]||\cdots||\mathbf{M}[i]^*||\ldots||\mathbf{M}[n]$.
- *PoW Correctness*. Let $\{T_i\}_{i\in[n']}$ denote the set of tags generated from $\mathbf{M}$. For any challenge Q and $\mathcal{P} \leftarrow \mathsf{PoWPrf}(\mathbf{M}, Q)$, we have the probability $\Pr[\mathsf{PoWVer}(\mathcal{P}, Q, \{T_i\}_{i\in Q}) = \mathsf{true}] = 1$.

3.4 Security Definition for UMLE

We formalize the security definitions for UMLE. We consider two traditional notions of MLE: (strong) tag consistency and privacy. We also formulate a new notion called "context hiding" for the Update protocol we introduce. They are defined via games played between a probabilistic polynomial-time adversary (PPT) adversary $\mathcal{A}$ and a challenger.

3.4.1 Strong tag Consistency

We follow the literature to define file-tag consistency [6] and block-tag consistency [10] for UMLE to provide security against "duplication-faking attack."[6] Note that the tag consistency notion given by Chen et al. [10] is not compatible with ours because BL-MLE and UMLE have different interfaces. Also, their notion only covers block-tag consistency. More importantly, in contrast to their claim [10, Theorem 6],[7] their scheme does not achieve their own notion of tag consistency (see Supplementary Material).

A UMLE scheme is said to achieve *strong file-tag consistency* if no PPT SFTC adversary $\mathcal{A}$ has a non-negligible advantage in the following strong file-tag consistency game:

- Setup: The challenger generates and sends $\mathcal{A}$ all the system parameter P.
- Output: Eventually, $\mathcal{A}$ outputs $(\mathbf{M}, \mathbf{C}')$. Let $T'_0 = \mathsf{M\text{-}TagGen}(\mathbf{C}')$, $T_0 = \mathsf{M\text{-}TagGen}(\mathsf{Enc}(\mathbf{M}))$, and $\mathbf{M}' = \mathsf{Dec}(\mathsf{M\text{-}KeyGen}(\mathbf{M}), \mathbf{C}')$. If $(T_0 = T'_0)$ and $(\mathbf{M} \neq \mathbf{M}')$, outputs 1; otherwise, outputs 0.

We define $\mathcal{A}$'s advantage $\mathbf{Adv}^{\mathcal{A}}_{\mathsf{SFTC}}(\lambda)$ as the probability that the above game outputs 1.

Definition 1 (File-Tag Consistency). *A* UMLE *scheme is* SFTC*-secure if, for any unpredictable block source $\mathcal{M}$ and any PPT* SFTC *adversary $\mathcal{A}$, the advantage in the strong file-tag consistency game, $\mathbf{Adv}^{\mathcal{A},\mathcal{M}}_{\mathsf{SFTC}}$, is negligible.*

We say that a UMLE scheme achieves *strong block-tag consistency* if no PPT SBTC adversary $\mathcal{A}$ has a non-negligible advantage in the strong block-tag consistency game below:

- Setup: The challenger generates and sends $\mathcal{A}$ all the system parameter P.
- Output: Eventually, $\mathcal{A}$ outputs $(\mathbf{M}, i, \mathbf{C}')$. Let $T'_i = \mathsf{B\text{-}TagGen}(\mathbf{C}'[i])$, $T_i = \mathsf{B\text{-}TagGen}(\mathsf{Enc}(\mathbf{M})[i])$, and $\mathbf{M}'[i] = \mathsf{Dec}(\mathsf{M\text{-}KeyGen}(\mathbf{M}), \mathbf{C}')[i]$. If $(T_i = T'_i)$ and $(\mathbf{M}[i] \neq \mathbf{M}'[i])$, outputs 1; or outputs 0 otherwise.[8]

We define $\mathcal{A}$'s advantage $\mathbf{Adv}^{\mathcal{A}}_{\mathsf{SBTC}}(\lambda)$ as the probability that the above game outputs 1.

Definition 2 (Block-Tag Consistency). *A* UMLE *scheme is* SBTC*-secure if, for any unpredictable block source $\mathcal{M}$ and any PPT* SBTC *adversary $\mathcal{A}$, the advantage in the strong block-tag consistency game, $\mathbf{Adv}^{\mathcal{A},\mathcal{M}}_{\mathsf{SBTC}}$, is negligible.*

We say that a UMLE scheme is strongly tag-consistent if it is *both* file-tag consistent and block-tag consistent.

3.4.2 Privacy

No MLE scheme can achieve the *de facto* standard of indistinguishability against chosen-plaintext attack (semantic security) [6], in which the adversary has complete freedom in choosing the plaintext messages. What we could achieve is a set of weakened privacy notions defined by Bellare et al. [6] for *unpredictable sources* (described in Section 2.2). In particular, the PRV-CDA notion [6] guarantees that encryption of unpredictable message must be indistinguishable from encryption of a random string of the same length.

We follow the spirit in existing works [6], [10] to define the privacy model[9] for UMLE, denoted by PRV-CDA-B*. We say that a UMLE scheme is secure under chosen-distribution attacks if no PPT adversary $\mathcal{A}$ has a non-negligible advantage in the PRV-CDA-B* game:

- Setup: $\mathcal{A}$ sends the description of an unpredictable source $\mathcal{M}$ to the challenger. The challenger then generates and sends $\mathcal{A}$ the system parameter P.
- Challenge: The challenger randomly picks $b \leftarrow \{0,1\}$. If $b = 0$, it runs $(\mathbf{M}_0, Z) \leftarrow \mathcal{M}(\lambda)$. If $b = 1$, it chooses $\mathbf{M}_1$ uniformly at random from $\{0,1\}^{|\mathbf{M}_0|}$. Set $\mathbf{M} = \mathbf{M}_b$. Let n be the number of blocks. For each $i \in [1, n]$, the challenger computes
  (1) the block keys $k_i \leftarrow \mathsf{B\text{-}KeyGen}(\mathbf{M}[i])$,
  (2) the ciphertext $\mathbf{C}[i] \leftarrow \mathsf{B\text{-}Enc}(k_i, \mathbf{M}[i])$, and
  (3) the encryption of k_i's: $\mathbf{C}[n+1]||\cdots||\mathbf{C}[n'] \leftarrow \mathsf{BK\text{-}Enc}(k_1, \ldots k_n)$.

Denote $\mathbf{C} = \mathbf{C}[1]||\cdots||\mathbf{C}[n]||\mathbf{C}[n+1]||\cdots||\mathbf{C}[n']$. The challenger also computes the file tag $T_0 \leftarrow \mathsf{M\text{-}TagGen}(\mathbf{C})$ and block tags $T_i \leftarrow \mathsf{B\text{-}TagGen}(\mathbf{C}[i])$ for each $i \in [1, n]$. Set $\mathbf{T} = \{T_0, T_1, \ldots, T_{n'}\}$.

6. The server wrongly believes that the plaintexts underlying two ciphertexts are the same (hence "fake duplication"). We refer readers to [6], [10] for more detailed discussions.

7. Chen et al. did not provide a full proof in their paper [10].

8. We are slightly abusing the notation. $\mathsf{Enc}(\mathbf{M})[i]$ should be understood as encrypting $\mathbf{M}$ first and then take the i-th block of the ciphertext. $\mathsf{Dec}(\mathsf{M\text{-}KeyGen}(\mathbf{M}), \mathbf{C}')[i]$ should be interpreted similarly.

9. Our notion differs from that of Chen et al. [10] due to different interfaces; e.g., our algorithms M-TagGen and B-TagGen take $\mathbf{C}$ as input but not $\mathbf{M}$, our Enc algorithm is split into two phases, etc.

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

Finally, the challenger gives auxiliary information Z, tags $\mathbf{T}$, and the ciphertext $\mathbf{C}$ to the adversary.

- Output: After receiving $(\mathbf{C}, \mathbf{T}, Z)$, the adversary outputs his guess b' and wins the game if $b' = b$.

We define $\mathcal{A}$'s advantage by

$$\mathbf{Adv}^{\mathcal{A},\mathcal{M}}_{\mathsf{PRV\text{-}CDA\text{-}B^*}}(\lambda) = |\Pr[b = b'] - 1/2|.$$

Definition 3 (Privacy). *A* UMLE *scheme is* PRV-CDA-B**-secure if for any unpredictable block source $\mathcal{M}$ and any PPT* PRV-CDA-B* *adversary $\mathcal{A}$, $\mathbf{Adv}^{\mathcal{A},\mathcal{M}}_{\mathsf{PRV\text{-}CDA\text{-}B^*}}$ is negligible.*

3.4.3 Context Hiding

We define a new security notion called "*context hiding*" which is specific to UMLE. Context hiding requires that the updated ciphertext is indistinguishable from a ciphertext encrypted from scratch to ensure that the update process does not reduce the level of privacy. Consider the following motivating scenario: Alice uploaded a file F under UMLE to the storage server. She performed some updates to sanitize sensitive information of the file before sharing it with Bob. Bob then uploads the sanitized version F' to the same storage server. By the requirement of deduplication system, Bob will be able to download Alice's encryption version of F' from the storage server later on. If UMLE is not context hiding, Bob might be able to infer some information of the original file F.

We say a UMLE scheme is context hiding if no adversary $\mathcal{A}$ has a non-negligible advantage in the CXH game below:

- Setup: The challenger generates and sends $\mathcal{A}$ the system parameter P.
- Challenge: $\mathcal{A}$ chooses two messages $\mathbf{M}_0$ and $\mathbf{M}_1$ that differs in one arbitrary single block (say the i-th block) of his choosing. Upon receiving $\mathbf{M}_0, \mathbf{M}_1$, the challenger examines whether $\mathbf{M}_0, \mathbf{M}_1 \in \mathrm{MsgSp}(\lambda)$ and whether $\mathbf{M}_0, \mathbf{M}_1$ differs in one block. If not, the challenger aborts; otherwise, it computes $\mathbf{C}_0 \leftarrow \mathsf{Enc}(\mathbf{M}_0)$ and $\mathbf{C}_1 \leftarrow \mathsf{Enc}(\mathbf{M}_1)$. Then it simulates an honest execution of Update. The simulated client uses $k_{\mathsf{mas}} \leftarrow \mathsf{M\text{-}KeyGen}(\mathbf{M}_1)$, i, and $\mathbf{M}_0[i]$ as inputs while the simulated server uses $\mathbf{C}_1$ as input. Let $\mathbf{C}'_1$ be the result. The simulator chooses a random bit b and sets the challenge ciphertext $\mathbf{C}^* = \mathbf{C}_0$ if $b = 0$; $\mathbf{C}^* = \mathbf{C}'_1$ otherwise.
- Output: Upon receiving $\mathbf{C}^*$, the adversary outputs its guess b'. The adversary wins the game if $b' = b$.

We refer to such an adversary $\mathcal{A}$ as a CXH adversary and define $\mathcal{A}$'s advantage by

$$\mathbf{Adv}^{\mathcal{A},\mathcal{M}}_{\mathsf{CXH}}(\lambda) = |\Pr[b = b'] - 1/2|.$$

Definition 4 (Context Hiding). *We say that a* UMLE *scheme is* CXH *if, for any* CXH *adversary $\mathcal{A}$, the advantage in the context hiding game, $\mathbf{Adv}^{\mathcal{A},\mathcal{M}}_{\mathsf{CXH}}$, is negligible.*

Note that we do not aim to hide the update pattern against the storage server since it is always revealed by the functional requirement of deduplication.

Statistical Indistinguishability. Our definition does not restrict the adversary to be polynomial time. This implies that the updated ciphertext must be statistically indistinguishable from one generated using Enc directly. If UMLE is deterministic (which is the case for our construction), this further implies that these two ciphertexts are identical.

The condition that the challenge messages $\mathbf{M}_0, \mathbf{M}_1$ only differ by 1 block is for definitional simplicity. By the transitivity of statistical indistinguishability, the ciphertext indistinguishability remains after polynomially many updates.

3.4.4 PoW Security

For PoW security, based on the bounded retrieval model (e.g., [21]), we consider the probability for an adversary to convince the server that it owns the entire file F while it only knows a bounded number of blocks of F. We say that a UMLE scheme is secure against an uncheatable chosen-distribution attack if no PPT adversary $\mathcal{A}$ has a non-negligible advantage in the UNC-CDA game below:

- Setup: The challenger $\mathcal{C}$ generates and sends $\mathcal{A}$ the system parameter P.
- Challenge: $\mathcal{A}$ sends $\mathcal{C}$ a valid source $\mathcal{M}$. $\mathcal{C}$ runs $(\mathbf{M}, Z) \leftarrow \mathcal{M}(\lambda)$ and sends the proof query Q and the auxiliary information Z to $\mathcal{A}$.
- Finally, $\mathcal{A}$ outputs $\mathcal{P}^*$. If $\mathsf{PoWVer}(\mathcal{P}^*, Q, \{T_i\}_{i \in Q})$ returns true, i.e., the proof $\mathcal{P}^*$ passes the verification, the challenger outputs 1; otherwise, outputs 0.

We define $\mathcal{A}$'s advantage $\mathbf{Adv}^{\mathcal{A},\mathcal{M}}_{\mathsf{UNC\text{-}CDA}}(\lambda)$ as the probability that the game outputs 1.

Definition 5. *A* UMLE *scheme is* UNC-CDA*-secure if, for any unpredictable block-source $\mathcal{M}$ and any polynomial time adversary $\mathcal{A}$, the advantage $\mathbf{Adv}^{\mathcal{A},\mathcal{M}}_{\mathsf{UNC\text{-}CDA}}(\lambda)$ is negligible.*

PoW security (Definition 5) appears to be orthogonal and complementary to tag consistency (Definitions 1 and 2). A tag-consistent (updatable BL-)MLE scheme may still be vulnerable to the attacks PoW security aims to defend (see Sections 2.4 and 3.2). On the other hand, PoW security assumes that the tags [1], [10] (or the "summary" [21]) are generated honestly by the challenger. Hence, PoW security cannot prevent tag inconsistency. As a concrete example, Chen et al.'s construction [10] satisfies PoW security but is not tag consistent (see Supplementary Material).

Although PoW provides extra security for the server that uses client-side deduplication, it does not prevent the client from learning side-channel leakage, e.g., whether the server already stored a certain file. Such leakage can be mitigated by an orthogonal line of research, e.g., [22].

3.5 Efficiency Requirements

We mandate the following efficiency requirements.

- For decrypting and updating arbitrarily long ciphertext, the local storage size ($|k_{\mathsf{mas}}|$) is $O(1)$.
- The ciphertext size only inflates by a multiplicative constant: $|\mathsf{Enc}(\mathbf{M})| \in O(|\mathbf{M}|)$.
- The computational and communicational complexities of Update and UpdateTag are $o(|\mathbf{M}|)$.
- The computational and communicational complexity of the (optional) PoW protocol is sublinear in $\mathbf{M}$.

Our efficiency requirements exclude trivial solutions, e.g.:

(1) applying file-level MLE on each block, and letting the concatenation of all block keys to be k_{mas};

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

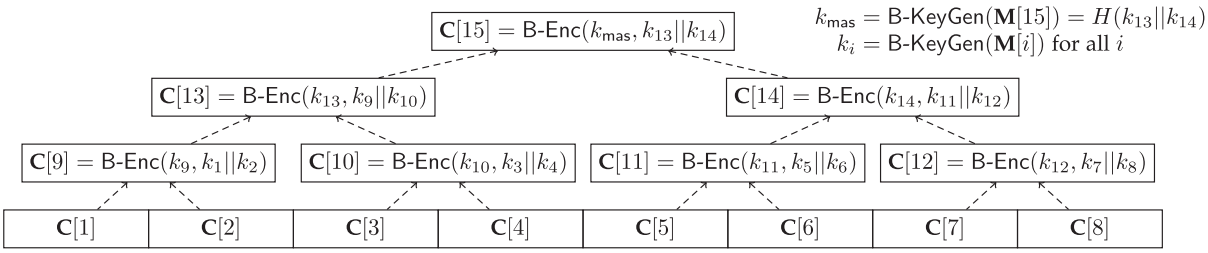

Fig. 1. The ciphertext of an 8-block file $\mathbf{M} = \mathbf{M}[1]||\cdots||\mathbf{M}[8]$ when block size B is double of key size λ, $\mathbf{M}[9] = k_1||k_2$, $\mathbf{M}[10] = k_3||k_4$, $\mathbf{M}[11] = k_5||k_6$, $\mathbf{M}[12] = k_7||k_8$, $\mathbf{M}[13] = k_9||k_{10}$, $\mathbf{M}[14] = k_{11}||k_{12}$, $\mathbf{M}[15] = k_{13}||k_{14}$

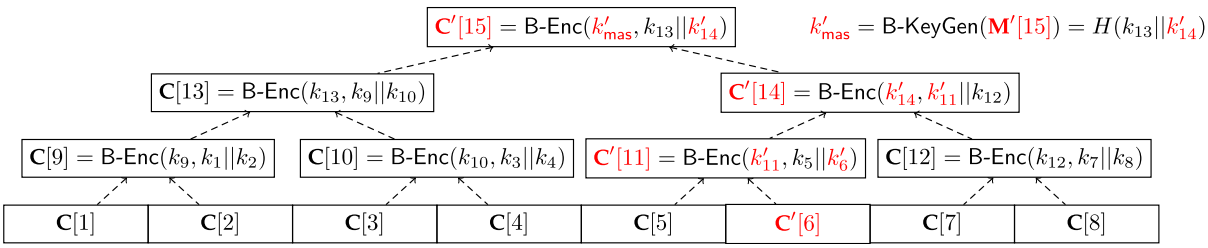

Fig. 2. Running Update to change $\mathbf{M}[6]$ to $\mathbf{M}'[6]$: only blocks $\mathbf{C}[6]$, $\mathbf{C}[11]$, $\mathbf{C}[14]$, and $\mathbf{C}[15]$, and k_{mas} are updated

(2) taking any (non-updatable) BL-MLE, the Update protocol simply recomputes KeyGen, Enc, and TagGen from scratch;
(3) verifying the PoW proof by processing the entire $\mathbf{M}$.

4 Proposed Construction

4.1 Intuition

We first describe the high-level idea of our construction. As far as we know, all existing MLE constructions follow the original paradigm of convergent encryption [2], [6]: derive the encryption key k_{mas} by hashing the *whole* file $\mathbf{M}$. Therefore, a single bit-flip in $\mathbf{M}$ is likely to result in a totally different master key k'_{mas}, and hence a completely different ciphertext. It seems that such a paradigm inherently forbids efficient update. We bypass this obstacle by recursively deriving k_{mas} *during* encryption.

In more details, we first derive the block keys for each block independently and encrypt each block. Then, we treat the concatenation of these block keys as a new plaintext message $\mathbf{M}'$ and apply MLE to encrypt it blockwise. This process is repeated until one single block can eventually accommodate the block keys generated in the previous step. The MLE key for this last block becomes the master key k_{mas}, which will be used by the Dec algorithm and the Update protocol. Also, as a result, the M-KeyGen algorithm is almost the same as the BK-Enc algorithm except that the outputs are different. We present the detailed steps of M-KeyGen and BK-Enc together in Algorithm 1 with differences highlighted in boxes.

The example in Fig. 1 illustrates how the ciphertext looks like when encrypting an 8-block message when a block can hold two keys (i.e., $B = 2\lambda$). The dashed arrows indicate the implicit relation between ciphertext blocks.

With our specific design, to change one plaintext block, the Update protocol (Fig. 3) only needs to change $\log(n)$ ciphertext blocks. (Fig. 2 shows a concrete example.) Updating the block tags is straightforward: recompute directly from updated ciphertext blocks. On the other hand, updating a file tag requires invoking IH.Update of the incremental hash by Bellare and Micciancio [15] $\log(n)$ times.

4.2 Detailed Description

Here we describe the details of our proposed construction. For the ease of presentation, we assume that the block size B is divisible by the key length $|k_{\text{mas}}| = \lambda$. We also assume that the number of blocks n is a power of B/λ. These restrictions are not essential (see Section 4.2.2).

- Setup: on input 1^λ, the algorithm outputs the description of a hash function $H : \{0,1\}^* \to \{0,1\}^\lambda$.
- KeyGen: on input $\mathbf{M} = \mathbf{M}[1]||\cdots||\mathbf{M}[n]$, run the following two sub-algorithms:
  - B-KeyGen: simply output $k_i = H(\mathbf{M}[i])$.
  - M-KeyGen: almost identical to BK-Enc (see Algorithm 1) except a few differences (boxed).
- Enc: on input $\mathbf{M}$, run the following sub-algorithms:
  - B-Enc: on input block message $\mathbf{M}[i]$ and the block key k_i, $1 \le i \le n$, return $\mathbf{C}[i] = \text{SKE.Enc}(k_i, \mathbf{M}[i])$.
  - BK-Enc: return $\mathbf{C}[n+1], \ldots, \mathbf{C}[n']$, see Algorithm 1.
- Dec: on input $(\mathbf{C}, k_{\text{mas}})$, run two sub-algorithms:
  - BK-Dec: return $k_1, \ldots, k_n$, see Algorithm 2.
  - B-Dec: on input block ciphertext $\mathbf{C}[i]$ and block key k_i, return $\mathbf{M}[i] = \text{SKE.Dec}(k_i, \mathbf{C}[i])$, for $1 \le i \le n$.
- TagGen: on input $\mathbf{C}$, run the following two algorithms to return file tag T_0 and block tags $T_1, \ldots, T_{n'}$:
  - B-TagGen: return $T_i = H(\mathbf{C}[i])$ for $1 \le i \le n'$.
  - M-TagGen: return $T_0 = \text{IH.Hash}(\mathbf{C})$.

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

User($k_{\mathsf{mas}}, i, \mathbf{M}[i]^*$)		**Server(C)**
$tmp = i$	$\xrightarrow{\quad i \quad}$	$tmp = i,\ idx = 1,\ \ell = \log_{B/\lambda} n$
for $j \in \{1, \ldots, \ell\}$		$st = 1,\ end = n,\ len = n$
$\quad idx_j = tmp \bmod B/\lambda$		**for** $j \in \{1, \ldots, \ell\}$
$\quad$ **if**$(idx_j == 0)$ **then** $idx_j = B/\lambda$		$\quad \mathbf{Buf}[idx] = \mathbf{C}[end + tmp]$
$\quad tmp = \lceil tmp \cdot \lambda/B \rceil$		$\quad tmp = \lceil tmp \cdot \lambda/B \rceil$
$varK = k_{\mathsf{mas}}$		$\quad len = len \cdot \lambda/B,\ \ idx = idx + 1$
for $j \in \{\ell, \ldots, 1\}$	$\xleftarrow{\ \mathbf{Buf}[1], \ldots, \mathbf{Buf}[\ell], \mathbf{C}[i]\ }$	$st = end + 1,\ \ end = st + len - 1$
$\quad \mathbf{K}[j]^* = \mathsf{SKE.Dec}(varK, \mathbf{Buf}[j])$		
$\quad$ Parse $\mathbf{K}[j]^*$ as $k_{j,1}\|\|k_{j,2}\|\|\cdots\|\|k_{j,B/\lambda}$		
$\quad varK = k_{j,idx_j}$		
$varK = H(\mathbf{M}[i]^*),\ \mathbf{C}[i]^* = \mathsf{SKE.Enc}(varK, \mathbf{M}[i]^*)$		
for $j \in \{1, \ldots, \ell\}$		
$\quad \mathbf{K}[j]^* = k_{j,1}\|\|\cdots\|\|k_{j,idx_j-1}\|\|varK\|\|k_{j,idx_j+1}\|\|\cdots\|\|k_{j,B/\lambda}$		
$\quad varK = H(\mathbf{K}[j]^*),\ \mathbf{Buf}[j]^* = \mathsf{SKE.Enc}(varK, \mathbf{K}[j]^*)$		
Output $k^*_{\mathsf{mas}} = varK$	$\xrightarrow{\ \mathbf{Buf}^*[1], \ldots, \mathbf{Buf}^*[\ell], \mathbf{C}[i]^*\ }$	Replace $\mathbf{Buf}[j]$ with $\mathbf{Buf}[j]^*\ \forall\, j$
		Replace $\mathbf{C}[i]^*$ with $\mathbf{C}[i]^*$

Fig. 3. Update protocol between user and server

Algorithm 1. BK-Enc $\boxed{\text{M-KeyGen}}$

Input: $k_1, \ldots, k_n$ $\boxed{M}$
Output: $\mathbf{C}[n+1], \ldots, \mathbf{C}[n']$ $\boxed{k_{\mathsf{mas}}}$
procedure
  Set $len = n, \ell = \log_{B/\lambda} n$ // namely, $n = (B/\lambda)^\ell$
  $\boxed{\text{For M-KeyGen, run B-KeyGen to get } k_1, \ldots, k_n}$
  Set $\mathbf{M}' = k_1\|\cdots\|k_n, idx = n+1$
  for $i \leftarrow \{1, \ldots, \ell\}$ **do**
    $len = len \cdot \lambda/B$
    Parse $\mathbf{M}'$ as blocks $\mathbf{M}'[1]\|\cdots\|\mathbf{M}'[len]$
    for $j \leftarrow \{1, \ldots, len\}$ **do**
      $k'_j = H(\mathbf{M}'[j])$
      $\mathbf{C}[idx] = \mathsf{SKE.Enc}(k'_j, \mathbf{M}'[j])$
      $idx = idx + 1$
    end for
    $\mathbf{M}' = k'_1\|\cdots\|k'_{len}$
    $k_{\mathsf{mas}} = H(\mathbf{M}')$
  end for
  return $\mathbf{C}[n+1], \ldots, \mathbf{C}[n']$ $\boxed{k_{\mathsf{mas}}}$
end procedure

The above describes the algorithms which are common in all MLE schemes. Below, we provide PoW related algorithms [10]. When a user shows a file tag T_0 that has been outsourced to the server, the server challenges this user to make sure that s/he indeed owns this file by using PoWVer to verify the proof generated by the user via PoWPrf.

- PoWPrf: Given a challenge Q, compute encrypted blocks $\mathbf{C}[i]$ and their block-tags T_i, $\forall i \in Q$. Output the proof $\Pi = \{T_i\}$.
- PoWVer: Given $\{T_i\}$, output 1 if they are the same as the records in the database; 0 otherwise.

Fig. 3 describes the Update protocol, which is the core part of our UMLE. It updates the remote ciphertext.

- Update: The client takes as inputs k_{mas}, the index i of the block to be updated, a plaintext block $\mathbf{M}[i]^*$; the server takes $\mathbf{C}$ as input. The steps are shown in Fig. 3.

Algorithm 2. BK-Dec

Input: $k_{\mathsf{mas}}, \mathbf{C}[n+1], \ldots, \mathbf{C}[n']$
Output: $k_1, \ldots, k_n$
procedure
  // Starts with decrypting $\mathbf{C}[n']$ by k_{mas}
  Set $len = 1, k_1 = k_{\mathsf{mas}}$
  Set $\ell = \log_{B/\lambda} n, st = end = n'$
  for $i \leftarrow \{1, \ldots, \ell\}$ **do**
    $len = len \cdot B/\lambda, idx = 0$
    // Decrypt $\mathbf{C}[st, end]$ to get $k_1, \ldots, k_{len}$
    for $j \leftarrow \{st, \ldots, end\}$ **do**
      $idx = idx + 1$
      $\mathbf{M}'[idx] = \mathsf{SKE.Dec}(k_{idx}, \mathbf{C}[j])$
    end for
    Parse $\mathbf{M}'[1]\|\cdots\|\mathbf{M}'[idx]$ as $k_1, \ldots, k_{len}$
    $end = st - 1, st = st - len$
  end for
  return $k_1, \ldots, k_n$
end procedure

Intuitively, Update partially decrypts the ciphertext $\mathbf{C}$ and then re-encrypts the affected parts. Recall the ciphertexts corresponding to the actual plaintexts are placed at the leaf level of the tree (Fig. 2). Changing one leaf affects all the nodes along the root-to-leaf path, but not the other paths. The client locally prepares the indices of those to-be-updated block keys (the first for-loop on the left in Fig. 3) before receiving all the ciphertext blocks along the path from the server. With the master key k_{mas}, the client can decrypt the blocks one by one by first decrypting the root ciphertext. This recovers the key k' for the next block so the

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

client can decrypt the next block using k', and so on (the second for-loop on the left). To re-encrypt, the client generates a new key for the updated leaf, replaces the old key at its parent node with the new key, re-encrypts the parent node, and so on, until all the affected blocks along the root-to-leaf path are re-encrypted (the third for-loop on the left).

In summary, Update executes a subset of the steps of BK-Dec and BK-Enc only. During Update, a new master key k'_{mas} is also generated because M-KeyGen algorithm is almost the same as BK-Enc. (See Section 4.2.1.) Note that the ciphertexts are generated in a hierarchical manner. Fig. 2 shows a concrete example of how the Update protocol will change the master key and the ciphertext.

Finally, UpdateTag updates the file tag and block tags.

- UpdateTag: given two versions of ciphertext $\mathbf{C}$ and $\mathbf{C}'$, let I be the set of indices i such that $\mathbf{C}[i] \neq \mathbf{C}'[i]$, run B-TagGen($\mathbf{C}'[i]$) for all $i \in I$ to obtain new block tags T'_i. Furthermore, set $T_{\mathsf{temp}} = T_0$, run $T_{\mathsf{temp}} \leftarrow \mathsf{IH.Update}(i, \mathbf{C}[i], \mathbf{C}'[i], T_{\mathsf{temp}})$ for all $i \in I$. Return the final updated file tag $T'_0 = T_{\mathsf{temp}}$.

4.2.1 Saving Time in BK-Enc

The M-KeyGen and BK-Enc algorithms are essentially the same, except the outputs are different. For the first time a file is ever encrypted, or any other situation where M-KeyGen is executed right before BK-Enc, we only execute M-KeyGen for once but also stores the output of $\mathbf{C}[n+1], \ldots, \mathbf{C}[n']$.

4.2.2 Flexible Number of Blocks

When the number of block n is not a power of B/λ, all algorithms work roughly in the same way. The only difference appears when encrypting or decrypting the last block within the second for-loop. This very last block may not contain B/λ block keys. We can apply efficiently invertible padding to fill the space so that the encryption and decryption algorithms can proceed normally.

4.3 Correctness and Efficiency

The correctness for *decryption*, *block-key retrieval*, and *tag* are all straightforward. The former two come from the correctness of the symmetric key encryption while the last one comes from the deterministic nature of all the tags.

Recall the efficiency requirements in Section 3.5. First, $|k_{\mathsf{mas}}| \in O(1)$ is trivial to see. For $|\mathsf{Enc}(\mathbf{M})| \in O(|\mathbf{M}|)$, note that the variable len is initialized with n in the outer loop, and decreases by a factor of B/λ (assuming $B \geq 2\lambda$) for each iteration. Therefore, the total number of additional ciphertexts is $n \cdot \lambda/B + n \cdot (\lambda/B)^2 + \cdots + 1 = \lceil \frac{(n-1)\lambda}{B-\lambda} \rceil \in O(n)$.[10] For the last requirement, it is straightforward to verify that the Update protocol only affects $\log(n)$ blocks due to the tree-like structure. (See Fig. 2 for a concrete example.) UpdateTag only involves recomputing block tag and executing IH.Update for each updated ciphertext block.

10. We stress that the multiplicative constant behind the big-oh notation is roughly $\lambda/(B-\lambda)$, which is very small in practice.

5 Performance Evaluation

We implemented the incremental hash function (IH.Hash, IH.Update) [15], which is used as the M-TagGen generation algorithm in our UMLE construction. We report its performance under different block size setting (in Supplementary Material). To the best of our knowledge, this evaluation is the first one in the literature.

We instantiate SKE in UMLE by `AES256_CTR` mode with a fixed initial vector (IV) and instantiate the hash function $H : \{0,1\}^* \rightarrow \{0,1\}^\lambda$ using `SHA256`. We compare the performance of our UMLE algorithms with (non-updatable) block-level MLE scheme [10] and (non-updatable) file-level MLE scheme [6]. Before our work, only a subset of the algorithms involving public-key operations was evaluated [10].

Our implementation uses `C` language, `OpenSSL` library (version 1.0.2g) implementation of `SHA256` and `AES256_CTR` mode, and Pairing-Based Cryptography (`PBC`) library[11] (version 0.5.14). Our experiment runs on a Linux machine (CentOS release 6.6 (Final)) with i7-3770/3.40GHz Quad Cores CPU and 8 GB RAM. We read the whole file into memory, then measure the CPU execution time of various algorithms, but omit the I/O time. All the reported numbers are averaged over 10 independent executions.

5.1 Comparing UMLE with BL-MLE [10]

Chen et al.'s BL-MLE construction [10] assumes the block space $\mathrm{BlSp}(\lambda)$ is $\{\mathbb{Z}_p\}^{\#s}$ and uses three hash functions $H_1 : \{0,1\}^* \rightarrow \mathbb{Z}_p$, $H_2 : \{\mathbb{Z}_p\}^{\#s} \rightarrow \mathbb{G}$, and $H_3 : \mathbb{G} \rightarrow \{\mathbb{Z}_p\}^{\#s}$. H_1 and H_2 are both instantiated by composing `SHA256` and `element_from_hash` (in `PBC`). H_3 is used to generate a one-time pad from the block key k_i to encrypt $\mathbf{M}[i]$ to $\mathbf{C}[i] = H_3(k_i) \oplus \mathbf{M}[i]$. We instantiate the encryption process by composing `SHA256` and `AES256_CTR` mode of encryption with a fixed IV, namely $\mathbf{C}[i] = \mathtt{AES256_CTR}(\mathtt{SHA256}(k_i), \mathbf{M}[i])$. For **M-TagGen**, **B-TagGen**, and **M-KeyGen**, we choose the Type-A curve (256-bit prime order group) of the `PBC` library, as the experiment of Chen et al. [10].

To have a fair comparison, we place functionally equivalent algorithms (albeit they may have different names and/or inputs) under the same column in Table 1. KeyGen and Enc in UMLE are merged into the KeyGen & Enc column because, in our construction, M-KeyGen (a sub-algorithm of KeyGen) and B-Enc (a sub-algorithm of Enc) are essentially the same except the outputs are different (see Section 4.2.1). We optimize our implementation by executing such overlapping codes only once in KeyGen and Enc. Hence, we only report the total execution time of these two algorithms.

In BL-MLE [10], the functionally equivalent algorithms of KeyGen & Enc in UMLE are **M-KeyGen**, **B-KeyGen**, and **Enc**. For a meaningful comparison with KeyGen & Enc, one should add the numbers in the three columns of **M-KeyGen**, **B-KeyGen**, and **Enc** in the BL-MLE sub-table.

Decryption in BL-MLE [10] requires the "retrieval" of the block keys from k_{mas} and tags T_i via **B-KeyRet**. In contrast, BK-Dec in UMLE computes the block keys from k_{mas} and ciphertext blocks $\mathbf{C}[n+1]|| \cdots ||\mathbf{C}[n']$. While BK-Dec and **B-KeyRet** have different input interfaces, their outputs are the same. So, we place them in the same column. The numbers

11. http://crypto.stanford.edu/pbc

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

TABLE 1
Computation Time (in ms) of BL-MLE and UMLE under a 256-bit Key, Different File Sizes $|\mathbf{M}|$ and Block Sizes B (Equivalent Algorithms under Different Names are Placed at the Same Column for Easy Comparison)

	$\|\mathbf{M}\|$	B	**M-KeyGen**	**B-KeyGen**	**Enc**	**B-KeyRet**	**Dec**	**B-TagGen**	**M-TagGen**
BL-MLE	2 MB	4 KB	5.552	1118.3	2.423	17886	2.095	17435	5.560
		8 KB	5.556	564.6	1.858	17357	1.582	17443	5.564
		16 KB	5.593	285.4	1.594	17196	1.308	17300	5.601
	4 MB	4 KB	11.121	2235.0	4.908	35684	3.688	35947	11.131
		8 KB	11.098	1138.0	3.828	35483	2.766	35044	11.109
		16 KB	11.078	569.7	3.219	34487	2.163	34317	11.088
	8 MB	4 KB	22.480	4497.4	9.629	71530	6.749	71990	22.492
		8 KB	22.401	2248.3	7.539	69418	4.937	69831	22.412
		16 KB	22.399	1139.9	6.464	68343	4.038	68721	22.410
UMLE	$\|\mathbf{M}\|$	B	KeyGen & Enc			BK-Dec	B-Dec	B-TagGen	M-TagGen
	2 MB	4 KB	6.946			0.013	1.141	5.710	1120.4
		8 KB	6.764			0.007	1.064	5.573	563.4
		16 KB	6.665			0.006	1.032	5.517	285.2
	4 MB	4 KB	13.480			0.023	1.956	11.401	2244.5
		8 KB	13.157			0.012	1.823	11.293	1129.3
		16 KB	12.995			0.008	1.775	11.145	569.3
	8 MB	4 KB	26.587			0.034	3.567	23.123	4487.0
		8 KB	25.929			0.019	3.377	22.759	2256.3
		16 KB	25.568			0.012	3.236	22.481	1137.9

reported in columns **B-KeyRet**/BK-Dec and **B-TagGen**/B-TagGen are the total time spent to generate *all* block keys/tags. The number of block keys/tags can be computed by dividing the file size $|\mathbf{M}|$ by the block size B.

From Table 1, we can see that UMLE outperforms BL-MLE in all columns except the last one (M-TagGen) for *all* combinations of file size and block size. Note that we omit algorithms which are only in BL-MLE but not in UMLE. (See Section 3.2.) Taking one particular setting ($|\mathbf{M}| = 2$ MB, $B = 16$ KB) as an example, our KeyGen & Enc algorithms in UMLE are over 40 times faster than **B-KeyGen** in BL-MLE (which by itself is not enough to realize the same functionality as KeyGen & Enc) due to its use of H_2 ("MapToPoint"). Our BK-Dec and B-TagGen algorithms under this setting are even over 2,800,000 times and over 3,000 times faster than their counterparts in BL-MLE. We note that both B-TagGen and M-TagGen are executed by the *storage server* in UMLE, while their counterparts **B-TagGen** (which requires pairing and exponentiation) and **M-TagGen** are executed by the *client* in BL-MLE. The relatively slower M-TagGen algorithm in UMLE does not introduce any burden on the (weaker) client, but the overwhelmingly long execution time of **B-TagGen** in BL-MLE could prevent its practical usage.

We further note that M-TagGen in UMLE can be made more efficient by choosing a larger block size (see Supplementary Material) and exploiting parallelism. If we treat UMLE as a block-level MLE only (without the update function), we can replace M-TagGen with just doing `SHA256` over the whole ciphertext. The resulting scheme will outperform BL-MLE [10] in every measure of computation time.

5.2 Comparing UMLE with MLE [6]

We implemented the convergent encryption version of MLE [6]. Following our UMLE implementation, we use `SHA256` as the hash function and `AES256_CTR` mode with a fixed IV as the symmetric-key encryption.

We can see from Table 2 that even in the worst case, the encryption algorithm of UMLE is only slower than MLE by less than 5 percent. The decryption algorithm (BK-Dec and B-Dec) in UMLE can be 30 percent slower than decryption of MLE, but the absolute difference is hardly noticeable because they can complete within 1s even for a 1 GB file.

5.3 Ciphertext Size and Update Cost of UMLE

Below we analyze the ciphertext size of our scheme under different block size settings. Let B be the block size, n be the number of plaintext blocks (so $|\mathbf{M}| = Bn$), and λ be the key length in bits. We define *the number of ciphertext blocks affected by one* Update *operation*, and that of *additional ciphertext blocks*, be ϕ and τ respectively. In Section 4.3, we derived $\tau = \lceil \frac{(n-1)\lambda}{B-\lambda} \rceil$. The total size of the ciphertext is $|\mathbf{M}| + B\tau = B(n+\tau)$. The update cost of UMLE for the storage server is dominated by executing IH.Update for $\phi = (1 + \lceil \log_{B/\lambda}(n) \rceil)$ times.

The cost of the client is merely decrypting ϕ ciphertext blocks and encrypting ϕ updated message blocks. Consider a large file of size 1 GB and $\lambda = 256$ (as in our experiments in this section), we calculate the ciphertext size and the update cost in Table 3. We can see that the ciphertext size is

TABLE 2
Computation Time (in ms) of MLE and UMLE

	$\|\mathbf{M}\|$		KeyGen	Enc	Dec
MLE	64 MB		179.7	22.3	22.2
	128 MB		359.8	44.1	44.0
	256 MB		718.9	87.9	87.7
	512 MB		1438.1	175.5	175.1
	1 GB		2872.8	349.8	349.5
UMLE	$\|\mathbf{M}\|$	B	KeyGen & Enc	BK-Dec	B-Dec
	64 MB	4 KB	209.9	0.182	26.30
		8 KB	204.5	0.088	24.42
		16 KB	201.5	0.059	23.34
	128 MB	4 KB	418.8	0.574	52.23
		8 KB	408.1	0.190	48.51
		16 KB	402.5	0.084	46.54
	256 MB	4 KB	837.2	1.153	104.09
		8 KB	815.5	0.534	97.14
		16 KB	805.7	0.169	92.78
	512 MB	4 KB	1668.5	1.965	207.71
		8 KB	1630.8	1.066	192.74
		16 KB	1608.8	0.526	186.16
	1 GB	4 KB	3336.3	3.627	459.10
		8 KB	3257.2	1.901	386.34
		16 KB	3211.6	1.080	370.41

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

TABLE 3
Ciphertext Size of a Large File ($|\mathbf{M}| = 1$ GB) under Different Block Sizes B (and Numbers of Message Blocks n)

B	n	B/λ	$1 + \lceil \log_{B/\lambda} n \rceil$	$\lceil \frac{(n-1)\lambda}{B-\lambda} \rceil$	Ciphertext size	Update time		Communication cost
						Server	Client	
4 KB	262144	128	4	2065	1032.07 MB	17.6 ms	0.124 ms	32 KB
8 KB	131072	256	4	515	1028.02 MB	17.6 ms	0.162 ms	64 KB
16 KB	65536	512	3	129	1026.02 MB	13.2 ms	0.194 ms	96 KB

only a few megabytes longer than the plaintext size. The update cost for the storage server is less than 20 ms, and the client takes less than 0.2 ms, significantly better than re-computing Enc from scratch.

5.4 Multi-Thread Implementation of UMLE

So far, all our experiments are conducted in a single-thread setting, without exploiting the power of parallelism in our UMLE construction. Nowadays, modern CPU cores often support multi-threading. All the algorithms in our UMLE construction, including the KeyGen algorithm, are highly parallelizable. This is in sharp contrast to existing BL-MLE or MLE construction, whose KeyGen is applying a cryptographic hash function on the plaintext. Typically, cryptographic hash functions that adopt the Merkle-Damgård construction or the sponge construction are *iterative*; namely, they cannot enjoy the benefit of multi-threading.

We implemented a multi-thread version of our Enc algorithm using the `pthread` library. Fig. 4 demonstrates the performance of Enc. We can see that UMLE encryption is faster than SHA256 in the multi-thread setting.

6 Related Work

6.1 Secure Deduplication and MLE

After the first formulation of MLE, Abadi et al. [7] additionally consider plaintext distributions which depend on the public parameter, but their construction is not known to be secure for correlated plaintexts. Bellare and Keelveedhi [8] achieve that by allowing interactive upload/download protocols using

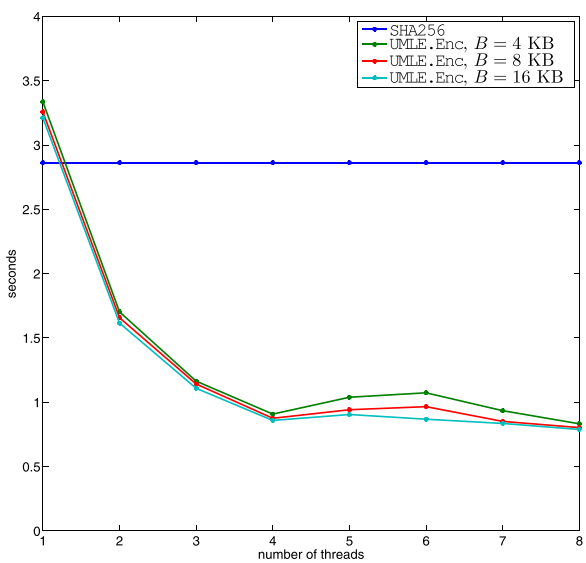

Fig. 4. Execution time of encrypting a 1 GB file using UMLE versus hashing a 1 GB file using SHA256

(heavyweight) fully-homomorphic encryption. They further propose the first secure deduplication system supporting incremental updates, but their scheme only deduplicates at the file level.

Armknecht et al. [24] consider a new research direction: how to make the customers of cloud storage service also economically benefit from deduplication technique. Canard et al. [25] introduce *deduplication consistency* that ensures an MLE scheme indeed deduplicates.

Liu et al. [26], [27] connect password-based authenticated key exchange (PAKE) with secure deduplication to better protect low-entropy files against offline-dictionary attack. However, PAKE involves public-key operations which are not that efficient. Moreover, they assume some clients voluntarily stay online after uploaded their files and participate in subsequent deduplication protocols (which involves PAKE) when some others upload. Such an assumption may not be practical. Stanek and Kencl [28] use threshold encryption to further encrypt MLE ciphertext. It provides higher security for unpopular files.

Recently, Kandele and Paul [29] proposed a new primitive *file-updatable message-locked encryption* (FMLE), with update cost linear in the length of the shortest suffix of the original file containing all modified content. The asymptotic update cost of UMLE is better than FMLE when the updated block is not the last $O(\log n)$ one, where n is the number of blocks. Jiang et al. [11] proposed a deduplication protocol supporting cross-user file-level deduplication, but it can only perform *intra-user* block-level deduplication. Liu et al. [30] added dynamic ownership management to updatable block-level deduplication system on encrypted data, but its update cost is $O(n)$.

Some researchers leverage additional server(s) to assist in deduplication and/or provide stronger security. For example, Sun et al. [31] introduce an additional key server for generating *randomized* block keys to protect low-entropy block messages; Ni et al. [32] make use of non-colluding fog nodes to detect and remove duplicated sensor data.

6.2 Plaintext-Check in Encryption/Encoding

All MLE is plaintext-checkable [33]: anyone can test if a ciphertext c corresponds to a given plaintext m. Plaintext-checkable encryption [33]/encoding [34] disallows duplication detection due to its unlinkability. Recently, Canard et al. [35] introduce a semi-honest party for obliviously generating plaintext-checkable (via pairing) encodings using its private encoding key.

7 Conclusion

We initiate the study of efficiently updatable block-level MLE. We conclude that encryption does not preclude deduplication

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.

or efficient ciphertext updates. Our construction allows updates, insertions, and deletions with (amortized) complexity of $O(\log |F|)$ for a file of size $|F|$. In contrast to existing non-updatable block-level MLE, our construction is pairing-free and satisfies tag consistency. Our comprehensive performance evaluation shows that its performance is comparable to non-updatable file-level MLE, and significantly better than existing non-updatable block-level MLE.

Acknowledgments

S.S.M. Chow is supported in part by the Early Career Award and General Research Funds (CUHK 14210217) of the Research Grants Council, Hong Kong. A preliminary version of this work [1] appeared in AsiaCCS 2017.

References

[1] Y. Zhao and S. S. M. Chow, "Updatable block-level message-locked encryption," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2017.
[2] J. R. Douceur, A. Adya, W. J. Bolosky, D. Simon, and M. Theimer, "Reclaiming space from duplicate files in a serverless distributed file system," in *Proc. Int. Conf. Distrib. Comput. Syst.*, 2002.
[3] A. Adya, W. J. Bolosky, M. Castro, G. Cermak, R. Chaiken, J. R. Douceur, J. Howell, J. R. Lorch, M. Theimer, and R. Wattenhofer, "FARSITE: Federated, available, and reliable storage for an incompletely trusted environment," in *Proc. Symp. Operating Syst. Des. Implementation*, 2002.
[4] P. Anderson and L. Zhang, "Fast and secure laptop backups with encrypted de-duplication," in *Proc. Int. Conf. Large Install. Syst. Admin.*, 2010, Art. no. 1–8.
[5] L. P. Cox, C. D. Murray, and B. D. Noble, "Pastiche: Making backup cheap and easy," in *Proc. Symp. Operating Syst. Des. Implementation*, 2002.
[6] M. Bellare, S. Keelveedhi, and T. Ristenpart, "Message-locked encryption and secure deduplication," *EUROCRYPT*, 2013.
[7] M. Abadi, D. Boneh, I. Mironov, A. Raghunathan, and G. Segev, "Message-locked encryption for lock-dependent messages," *CRYPTO*, 2013.
[8] M. Bellare and S. Keelveedhi, "Interactive message-locked encryption and secure deduplication," in *Proc. IACR Int. Workshop Public Key Cryptography*, 2015.
[9] D. T. Meyer and W. J. Bolosky, "A study of practical deduplication," *ACM Trans. Storage*, vol. 7, no. 4, 2012, Art. no. 14.
[10] R. Chen, Y. Mu, G. Yang, and F. Guo, "BL-MLE: block-level message-locked encryption for secure large file deduplication," *IEEE Trans. Inf. Forensics Secur.*, vol. 10, no. 12, pp. 2643–2652, Dec. 2015.
[11] S. Jiang, T. Jiang, and L. Wang, "Secure and efficient cloud data deduplication with ownership management," *IEEE Trans. Serv. Comput.*
[12] A. S. Szalay, J. Gray, A. Thakar, P. Z. Kunszt, T. Malik, J. Raddick, C. Stoughton, and J. vandenBerg, "The SDSS skyserver: Public access to the sloan digital sky server data," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2002.
[13] K. Jin and E. L. Miller, "The effectiveness of deduplication on virtual machine disk images," in *Proc. SYSTOR: The Israeli Exp. Syst. Conf.*, 2009, Art. no. 7.
[14] M. Bellare, O. Goldreich, and S. Goldwasser, "Incremental cryptography: The case of hashing and signing," *CRYPTO*, 1994.
[15] M. Bellare and D. Micciancio, "A new paradigm for collision-free hashing: Incrementality at reduced cost," *EUROCRYPT*, 1997.
[16] M. Bellare, O. Goldreich, and S. Goldwasser, "Incremental cryptography and application to virus protection," in *Proc. Annu. ACM Symp. Theory Comput.*, 1995.
[17] I. Mironov, O. Pandey, O. Reingold, and G. Segev, "Incremental deterministic public-key encryption," *EUROCRYPT*, 2012.
[18] S. Kamara, C. Papamanthou, and T. Roeder, "Dynamic searchable symmetric encryption," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2012.
[19] R. W. F. Lai and S. S. M. Chow, "Forward-secure searchable encryption on labeled bipartite graphs," in *Proc. Int. Conf. Appl. Cryptography Netw. Secur.*, 2017.
[20] Q. Wang, M. He, M. Du, S. S. M. Chow, R. W. F. Lai, and Q. Zou, "Searchable encryption over feature-rich data," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 3, pp. 496–510, May/Jun. 2018.
[21] S. Halevi, D. Harnik, B. Pinkas, and A. Shulman-Peleg, "Proofs of ownership in remote storage systems," in *Proc. ACM Conf. Comput. Commun. Secur.*, 2011.
[22] Y. Zhao and S. S. M. Chow, "Towards proofs of ownership beyond bounded leakage," in *Proc. Int. Conf. Provable Secur.*, 2016.
[23] F. Armknecht, C. Boyd, G. T. Davies, K. Gjøsteen, and M. Toorani, "Side channels in deduplication: Trade-offs between leakage and efficiency," in *Proc. ACM Asia Conf. Comput. Commun. Secur.*, 2017.
[24] F. Armknecht, J. Bohli, G. O. Karame, and F. Youssef, "Transparent data deduplication in the cloud," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015.
[25] S. Canard, F. Laguillaumie, and M. Paindavoine, "Verifiable message-locked encryption," in *Proc. Int. Conf. Cryptology Netw. Secur.*, 2016.
[26] J. Liu, N. Asokan, and B. Pinkas, "Secure deduplication of encrypted data without additional independent servers," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2015.
[27] J. Liu, L. Duan, Y. Li, and N. Asokan, "Secure deduplication of encrypted data: Refined model and new constructions," in *Proc. Cryptographers' Track RSA Conf.*, 2018.
[28] J. Stanek and L. Kencl, "Enhanced secure thresholded data deduplication scheme for cloud storage," *IEEE Trans. Dependable Secure Comput.*, vol. 15, no. 4, pp. 694–707, Jul./Aug. 2018.
[29] S. Kandele and S. Paul, "Message-locked encryption with file update," in *Proc. Int. Conf. Appl. Cryptography Netw. Secur.*, 2018.
[30] M. Liu, C. Yang, Q. Jiang, X. Chen, J. Ma, and J. Ren, "Updatable block-level deduplication with dynamic ownership management on encrypted data," in *Proc. IEEE Int. Conf. Commun.*, 2018.
[31] W. Sun, N. Zhang, W. Lou, and Y. T. Hou, "Tapping the potential: Secure chunk-based deduplication of encrypted data for cloud backup," in *Proc. IEEE Conf. Commun. Netw. Secur.*, 2018.
[32] J. Ni, K. Zhang, Y. Yu, X. Lin, and X. S. Shen, "Providing task allocation and secure deduplication for mobile crowdsensing via fog computing," *IEEE Trans. Dependable Secure Comput.*
[33] S. Canard, G. Fuchsbauer, A. Gouget, and F. Laguillaumie, "Plaintext-checkable encryption," in *Proc. Cryptographers' Track RSA Conf.*, 2012.
[34] Y. Zhao and S. S. M. Chow, "Privacy preserving collaborative filtering from asymmetric randomized encoding," in *Proc. Int. Conf. Financial Cryptography Data Secur.*, 2015.
[35] S. Canard, D. Pointcheval, Q. Santos, and J. Traoré, "Privacy-preserving plaintext-equality of low-entropy inputs," in *Proc. Int. Conf. Appl. Cryptography Netw. Secur.*, 2018.

Yongjun Zhao received the BEng degree (with first class honour) and the PhD degree from the Department of Information Engineering, Chinese University of Hong Kong. He is currently doing post-doctoral research in the, Chinese University of Hong Kong. His main research interests include applied cryptography, cryptographic cloud storage, privacy-enhancing technologies, and blockchain. He served on the program committees of many conferences including CANS, InsCrypt, NSS, SECRYPT, SCC, and SecureComm. He is a member of the IEEE.

Sherman S. M. Chow received the PhD degree from New York University. He is an associate professor with Chinese University of Hong Kong. He was a research fellow with the University of Waterloo. His interest is applied cryptography. He has published in AsiaCrypt, CCS, EuroCrypt, ITCS, NDSS, and Usenix Security. He served on the program committees of many conferences including AsiaCrypt, CCS, Crypto, ESORICS, Financial Crypt, ICDCS, Infocom, PETS, PKC, and TheWeb. He is an editor of the *IEEE Transactions on Information Forensics and Security (TIFS)*. He is a member of the IEEE.

▷ **For more information on this or any other computing topic, please visit our Digital Library at** www.computer.org/csdl.

Authorized licensed use limited to: Nanyang Technological University. Downloaded on September 15,2022 at 18:45:49 UTC from IEEE Xplore. Restrictions apply.